BLOCKSEC

Security Audit Report for onchain-solana

Date: September 23, 2025 **Version:** 1.0

Contact: contact@blocksec.com

Contents

Chapter 1 Introduction **1**
1.1 About Target Contracts . . . 1
1.2 Disclaimer . . . 1
1.3 Procedure of Auditing . . . 2
1.3.1 Security Issues . . . 2
1.3.2 Additional Recommendation . . . 2
1.4 Security Model . . . 3

Chapter 2 Findings **4**
2.1 Security Issue . . . 4
2.1.1 Lack of nonce check during swap . . . 4
2.2 Recommendation . . . 5
2.2.1 Improper logic in function `transfer_between_pdas()` . . . 5
2.2.2 Inconsistent logged amount during transfer . . . 9
2.2.3 The value of `amount_in` may be inconsistent with the actual amount of tokens used in the swap . . . 11
2.2.4 Lack of state checks in functions `pause()` and `unpause()` . . . 12
2.3 Note . . . 13
2.3.1 Potential centralization risks . . . 13
2.3.2 Ensure off-chain logic is trusted . . . 13

Report Manifest

Item	Description
Client	blcokchainbg
Target	onchain-solana

Version History

Version	Date	Description
1.0	September 23, 2025	First release

Signature

Digitally signed by: BlockSec (https://blocksec.com)

SHA1 digest of public key: BAB106D3708BC6DE0963CE05E9478C24B326AE1C

Date: 2026-03-30 03:42:05+0000

About BlockSec BlockSec focuses on the security of the blockchain ecosystem and collaborates with leading DeFi projects to secure their products. BlockSec is founded by top-notch security researchers and experienced experts from both academia and industry. They have published multiple blockchain security papers in prestigious conferences, reported several zero-day attacks of DeFi applications, and successfully protected digital assets that are worth more than 14 million dollars by blocking multiple attacks. They can be reached at Email, Twitter and Medium.

Chapter 1 Introduction

1.1 About Target Contracts

Information	Description
Type	Smart Contract
Language	Rust
Approach	Semi-automatic and manual verification

The target of this audit is the code repository [1] of onchain-solana of blcokchainbg.

The protocol is deployed on Solana and enables token swaps by routing each trade through a single, supported aggregator. Aggregators are whitelisted in configuration, and only accounts holding the caller role may invoke the swap path. Operator/owner roles manage configuration adding or removing aggregators, pausing the program, and maintaining authority PDAs. PDAs act as token authorities for secure CPI into SPL and Token-2022 programs, while min-out checks enforce slippage limits. Administrative tooling covers emergency withdrawals, inter-PDA transfers, and hot-wallet funding.

Note this audit only focuses on the smart contracts in the following directories/files:

- programs/onchain_solana/src/*

Other files are not within the scope of the audit. Additionally, all dependencies of the smart contracts within the audit scope are considered reliable in terms of both functionality and security, and are therefore not included in the audit scope.

The auditing process is iterative. Specifically, we would audit the commits that fix the discovered issues. If there are new issues, we will continue this process. The commit SHA values during the audit are shown in the following table. Our audit report is responsible for the code in the initial version (`Version 1`), as well as new code (in the following versions) to fix issues in the audit report. Code prior to and including the baseline version (`Version 0`), where applicable, is outside the scope of this audit and assumes to be reliable and secure.

Project	Version	Commit Hash
onchain-solana	`Version 1`	`bf42284456257d3d8fc455964cc5637eac0777c5`

1.2 Disclaimer

This audit report does not constitute investment advice or a personal recommendation. It does not consider, and should not be interpreted as considering or having any bearing on, the potential economics of a token, token sale or any other product, service or other asset. Any entity should not rely on this report in any way, including for the purpose of making any decisions to buy or sell any token, product, service or other asset.

This audit report is not an endorsement of any particular project or team, and the report does not guarantee the security of any particular project. This audit does not give any war-

[1] https://github.com/blcokchainbg/onchain-solana

ranties on discovering all security issues of the smart contracts, i.e., the evaluation result does not guarantee the nonexistence of any further findings of security issues. As one audit cannot be considered comprehensive, we always recommend proceeding with independent audits and a public bug bounty program to ensure the security of smart contracts.

The scope of this audit is limited to the code mentioned in Section 1.1. Unless explicitly specified, the security of the language itself (e.g., the solidity language), the underlying compiling toolchain and the computing infrastructure are out of the scope.

1.3 Procedure of Auditing

We perform the audit according to the following procedure.

- **Vulnerability Detection** We first scan smart contracts with automatic code analyzers, and then manually verify (reject or confirm) the issues reported by them.
- **Semantic Analysis** We study the business logic of smart contracts and conduct further investigation on the possible vulnerabilities using an automatic fuzzing tool (developed by our research team). We also manually analyze possible attack scenarios with independent auditors to cross-check the result.
- **Recommendation** We provide some useful advice to developers from the perspective of good programming practice, including gas optimization, code style, and etc.

We show the main concrete checkpoints in the following.

1.3.1 Security Issues

* Access control
* Permission management
* Whitelist and blacklist mechanisms
* Initialization consistency
* Improper use of the proxy system
* Reentrancy
* Denial of Service (DoS)
* Untrusted external call and control flow
* Exception handling
* Data handling and flow
* Events operation
* Error-prone randomness
* Oracle security
* Business logic correctness
* Semantic and functional consistency
* Emergency mechanism
* Economic and incentive impact

1.3.2 Additional Recommendation

* Gas optimization

* Code quality and style

Note *The previous checkpoints are the main ones. We may use more checkpoints during the auditing process according to the functionality of the project.*

1.4 Security Model

To evaluate the risk, we follow the standards or suggestions that are widely adopted by both industry and academy, including OWASP Risk Rating Methodology [2] and Common Weakness Enumeration [3]. The overall *severity* of the risk is determined by *likelihood* and *impact*. Specifically, likelihood is used to estimate how likely a particular vulnerability can be uncovered and exploited by an attacker, while impact is used to measure the consequences of a successful exploit.

In this report, both likelihood and impact are categorized into two ratings, i.e., *high* and *low* respectively, and their combinations are shown in Table 1.1.

Table 1.1: Vulnerability Severity Classification

Impact	Likelihood: *High*	Likelihood: *Low*
High	High	Medium
Low	Medium	Low

Accordingly, the severity measured in this report are classified into three categories: **High**, **Medium**, **Low**. For the sake of completeness, **Undetermined** is also used to cover circumstances when the risk cannot be well determined.

Furthermore, the status of a discovered item will fall into one of the following five categories:

- **Undetermined** No response yet.
- **Acknowledged** The item has been received by the client, but not confirmed yet.
- **Confirmed** The item has been recognized by the client, but not fixed yet.
- **Partially Fixed** The item has been confirmed and partially fixed by the client.
- **Fixed** The item has been confirmed and fixed by the client.

[2] https://owasp.org/www-community/OWASP_Risk_Rating_Methodology

[3] https://cwe.mitre.org/

Chapter 2 Findings

In total, we found **one** potential security issue. Besides, we have **four** recommendations and **two** notes.

- Low Risk: 1
- Recommendation: 4
- Note: 2

ID	Severity	Description	Category	Status
1	Low	Lack of nonce check during swap	Security Issue	Confirmed
2	-	Improper logic in function `transfer_between_pdas()`	Recommendation	Confirmed
3	-	Inconsistent logged amount during transfer	Recommendation	Confirmed
4	-	The value of `amount_in` may be inconsistent with the actual amount of tokens used in the swap	Recommendation	Confirmed
5	-	Lack of state checks in functions `pause()` and `unpause()`	Recommendation	Confirmed
6	-	Potential centralization risks	Note	-
7	-	Ensure off-chain logic is trusted	Note	-

The details are provided in the following sections.

2.1 Security Issue

2.1.1 Lack of nonce check during swap

Severity Low

Status Confirmed

Introduced by `Version 1`

Description The function `execute_swap()` allows a privileged `caller` to swap assets from a designated token account through a whitelisted aggregator. While the order parameters include `order_id`, `amount_in`, `min_amount_out`, and `deadline`, there is no mechanism to ensure that an order is uniquely executed. Without a nonce or replay protection, the same order data can be reused multiple times, allowing repeated swaps on the same order.

```
pub fn process(
    &mut self,
    from_pda_index: u8,
    to_pda_index: u8,
    order: OrderInfo,
    data: Vec<u8>,
    remaining_accounts: &[AccountInfo<'info>],
) -> Result<()> {
    ExecuteSwap::is_valid_order(&order)?;

```

```
        let recipient_before_balance = self.to_token_ata.amount;

        self.call_aggregator(from_pda_index, data, remaining_accounts)?;

        self.to_token_ata.reload()?;

        let recipient_after_balance = self.to_token_ata.amount;

        let amount_received = recipient_after_balance
            .checked_sub(recipient_before_balance)
            .ok_or(ProxyError::UnexpectedBalanceDecrease)?;

        msg!("toToken received: {}", amount_received);
        msg!("minAmountOut: {}", order.min_amount_out);

        require!(
            amount_received >= order.min_amount_out,
            ProxyError::InsufficientAmountOut
        );

        emit!(SwapExecuted {
            caller: self.signer.key(),
            aggregator: self.aggregator_program.key(),
            amount_in: order.amount_in,
            amount_out: amount_received,
            order_id: order.order_id,
            timestamp: Clock::get()?.unix_timestamp,
        });

        Ok(())
    }
```

Listing 2.1: programs/onchain_solana/src/instructions/execute_swap.rs

Impact The same order can be executed repeatedly, leading to unintended multiple swaps and potential loss of funds.

Suggestion Introduce a nonce or replay protection mechanism by tracking and invalidating previously used orders.

Feedback from the project This issue has low impact and does not affect user facing functionality, the team will address it in the next upgrade.

2.2 Recommendation

2.2.1 Improper logic in function `transfer_between_pdas()`

Status Confirmed

Introduced by `Version 1`

Description The function `transfer_between_pdas()` allows the protocol `owner` to transfer tokens between accounts. However, it determines whether the transfer involves `SOL` based on

the `token_mint` parameter, while the actual transfer logic operates on the `token_mint` derived from the `Context`. This inconsistency may affect correct execution of the function.

The same issue also exists in the function `transfer_to_hot_wallet()`.

```
pub fn process(
    &mut self,
    from_pda_index: u8,
    to_pda_index: u8,
    amount: u64,
    token_mint: Option<Pubkey>,
) -> Result<()> {
    require!(amount > 0, ProxyError::InvalidAmount);

    require!(from_pda_index != to_pda_index, ProxyError::InvalidTransfer);

    let is_sol_transfer = token_mint.is_none();

    if is_sol_transfer {
        self.transfer_sol(from_pda_index, amount)?;
    } else {
        self.transfer_token(from_pda_index, amount)?;
    }

    emit!(PdaTransfer {
        from_pda_index,
        to_pda_index,
        token_mint,
        amount,
        operator: self.signer.key(),
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}
```

Listing 2.2: programs/onchain_solana/src/instructions/transfer_between_pdas.rs

```
pub fn process(
    &mut self,
    pda_index: u8,
    destination_type: WalletType,
    amount: u64,
    token_mint: Option<Pubkey>,
) -> Result<()> {
    require!(amount > 0, ProxyError::InvalidAmount);

    let is_sol_transfer = token_mint.is_none();

    if is_sol_transfer {
        self.transfer_sol(pda_index, amount)?;
    } else {
        self.transfer_token(pda_index, amount)?;
    }
```

```rust

        emit!(FundsTransferred {
            operator: self.signer.key(),
            from_pda_index: pda_index,
            destination_type,
            destination: self.destination.key(),
            token_mint,
            amount,
            timestamp: Clock::get()?.unix_timestamp,
        });

        Ok(())
    }
```

Listing 2.3: programs/onchain_solana/src/instructions/transfer_to_hot_wallet.rs

```rust
pub struct TransferBetweenPdas<'info> {
    #[account(
        mut,
        owner = system_program::ID @ProxyError::OwnerSystemProgramID,
        constraint = config_data.is_operator(signer.key()) @ProxyError::InvalidOperator,
    )]
    pub signer: Signer<'info>,

    #[account(
        seeds = [ONCHAINPROXY_CONFIG_SEED],
        bump,
        constraint = !config_data.paused @ProxyError::Paused,
    )]
    pub config_data: Account<'info, ConfigDataState>,

    #[account(
        mut,
        seeds = [ONCHAINPROXY_AUTH_PDA_SEED, &[from_pda_index]],
        bump,
    )]
    /// CHECK: address is the source PDA account
    pub from_pda: AccountInfo<'info>,

    #[account(
        mut,
        seeds = [ONCHAINPROXY_AUTH_PDA_SEED, &[to_pda_index]],
        bump
    )]
    /// CHECK: address is the destination PDA account
    pub to_pda: AccountInfo<'info>,

    pub token_mint: Option<InterfaceAccount<'info, MintInterface>>,

    #[account(
        mut,
        associated_token::mint = token_mint,
        associated_token::authority = from_pda,
```

```
        associated_token::token_program = token_program,
    )]
    pub from_token_account: Option<InterfaceAccount<'info, TokenAccountInterface>>,

    #[account(
        init_if_needed,
        payer = signer,
        associated_token::mint = token_mint,
        associated_token::authority = to_pda,
        associated_token::token_program = token_program,
    )]
    pub to_token_account: Option<InterfaceAccount<'info, TokenAccountInterface>>,

    pub token_program: Option<Interface<'info, TokenInterface>>,
    pub associated_token_program: Option<Program<'info, AssociatedToken>>,
    pub system_program: Program<'info, System>,
}
```

Listing 2.4: programs/onchain_solana/src/instructions/transfer_between_pdas.rs

```
pub struct TransferToHotWallet<'info> {
    #[account(
        mut,
        owner = system_program::ID @ProxyError::OwnerSystemProgramID,
        constraint = config_data.is_operator(signer.key()) @ProxyError::InvalidOperator,
    )]
    pub signer: Signer<'info>,

    #[account(
        seeds = [ONCHAINPROXY_CONFIG_SEED],
        bump,
        constraint = !config_data.paused @ProxyError::Paused,
    )]
    pub config_data: Account<'info, ConfigDataState>,

    #[account(
        mut,
        seeds = [ONCHAINPROXY_AUTH_PDA_SEED, &[pda_index]],
        bump,
    )]
    /// CHECK: address is the source PDA account
    pub from_pda: AccountInfo<'info>,

    #[account(
        mut,
        constraint = destination.key() == config_data.get_hot_wallet(destination_type) @ProxyError
            ::InvalidDestination,
    )]
    /// CHECK: address is validated hot wallets(onchain hot/spot hot)
    pub destination: AccountInfo<'info>,

    // Optional accounts for SPL token transfers
    pub token_mint: Option<InterfaceAccount<'info, MintInterface>>,
```

```rust

    #[account(
        mut,
        associated_token::mint = token_mint,
        associated_token::authority = from_pda,
        associated_token::token_program = token_program,
    )]
    pub from_token_account: Option<InterfaceAccount<'info, TokenAccountInterface>>,

    #[account(
        init_if_needed,
        payer = signer,
        associated_token::mint = token_mint,
        associated_token::authority = destination,
        associated_token::token_program = token_program,
    )]
    pub to_token_account: Option<InterfaceAccount<'info, TokenAccountInterface>>,

    pub token_program: Option<Interface<'info, TokenInterface>>,
    pub associated_token_program: Option<Program<'info, AssociatedToken>>,
    pub system_program: Program<'info, System>,
}
```

Listing 2.5: programs/onchain_solana/src/instructions/transfer_to_hot_wallet.rs

Suggestion Revise the logic accordingly.

Feedback from the project This function is a privileged function, the team enforces parameter consistency via off chain server validation.

2.2.2 Inconsistent logged amount during transfer

Status Confirmed

Introduced by `Version 1`

Description The functions `transfer_sol()` and `transfer_token()` both handle transfers but log amounts in different formats. In function `transfer_sol()`, the logged amount is scaled to human-readable units, while in function `transfer_token()` it is logged in raw token units including decimals. This inconsistency can confuse operators and complicate monitoring of events.

```rust
    fn transfer_sol(&mut self, pda_index: u8) -> Result<()> {
        let amount = self.token_auth_pda.lamports();
        require!(amount > 0, ProxyError::InsufficientBalance);

        self.config_data.transfer_sol_with_token_auth_pda(
            &self.token_auth_pda,
            &self.vault,
            amount,
            pda_index,
            TransferType::Emergency,
        )?;

```

```
    msg!(
        "Emergency withdraw: {} SOL from PDA {} to vault",
        amount as f64 / 1_000_000_000.0,
        self.token_auth_pda.key()
    );

    emit!(EmergencyWithdrawEvent {
        operator: self.signer.key(),
        vault: self.vault.key(),
        amount,
        is_sol: true,
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}
```

Listing 2.6: programs/onchain_solana/src/instructions/emergency_withdraw.rs

```
fn transfer_token(&mut self, pda_index: u8) -> Result<()> {
    let token_mint = self.token_mint.as_ref().unwrap();
    let from_token_ata = self.program_token_ata.as_ref().unwrap();
    let to_token_ata = self.vault_token_ata.as_ref().unwrap();
    let token_program = self.token_program.as_ref().unwrap();

    let amount = from_token_ata.amount;
    require!(amount > 0, ProxyError::InsufficientBalance);

    self.config_data.transfer_token_with_token_auth_pda(
        &from_token_ata.to_account_info(),
        &to_token_ata.to_account_info(),
        &token_mint.to_account_info(),
        &self.token_auth_pda,
        &token_program.to_account_info(),
        amount,
        token_mint.decimals,
        pda_index,
        TransferType::Emergency,
        &self.config_data.vault,
    )?;

    msg!(
        "Emergency withdraw: {} tokens from PDA {} to vault",
        amount,
        self.token_auth_pda.key()
    );

    emit!(EmergencyWithdrawEvent {
        operator: self.signer.key(),
        vault: self.vault.key(),
        amount,
        is_sol: false,
        timestamp: Clock::get()?.unix_timestamp,
```

```
    });

    Ok(())
}
```

Listing 2.7: programs/onchain_solana/src/instructions/emergency_withdraw.rs

Suggestion Revise the logic to ensure the consistency.

Feedback from the project The team will adapt off chain logic to the on chain event logs to ensure that on chain functionality executes as intended.

2.2.3 The value of `amount_in` may be inconsistent with the actual amount of tokens used in the swap

Status Confirmed

Introduced by `Version 1`

Description In the `execute_swap()` instruction, the caller can execute the swap through whitelisted aggregators by `CPI`. After the swap, the contract will emit an event to log the information of the swap execution. However, the value of `amount_in` in the event may not be consistent with the actual amount of tokens used in the swap. It is recommended to record the actual value of `amount_in` to eliminate any ambiguity.

```
pub fn process(
    &mut self,
    from_pda_index: u8,
    to_pda_index: u8,
    order: OrderInfo,
    data: Vec<u8>,
    remaining_accounts: &[AccountInfo<'info>],
) -> Result<()> {
    ExecuteSwap::is_valid_order(&order)?;

    let recipient_before_balance = self.to_token_ata.amount;

    self.call_aggregator(from_pda_index, data, remaining_accounts)?;

    self.to_token_ata.reload()?;

    let recipient_after_balance = self.to_token_ata.amount;

    let amount_received = recipient_after_balance
        .checked_sub(recipient_before_balance)
        .ok_or(ProxyError::UnexpectedBalanceDecrease)?;

    msg!("toToken received: {}", amount_received);
    msg!("minAmountOut: {}", order.min_amount_out);

    require!(
        amount_received >= order.min_amount_out,
        ProxyError::InsufficientAmountOut
```

```
    );

    emit!(SwapExecuted {
        caller: self.signer.key(),
        aggregator: self.aggregator_program.key(),
        amount_in: order.amount_in,
        amount_out: amount_received,
        order_id: order.order_id,
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}
```

Listing 2.8: programs/onchain_solana/src/instructions/execute_swap.rs

Suggestion Record the actual value of `amount_in`.

Feedback from the project The team will adapt off chain logic to the on chain event logs to ensure that on chain functionality executes as intended.

2.2.4 Lack of state checks in functions `pause()` and `unpause()`

Status Confirmed

Introduced by `Version 1`

Description In the protocol, an `operator` can pause the protocol via function `pause()`, and the `owner` can resume it via function `unpause()`. However, neither function checks the current runtime state. Concretely, invoking the function `pause()` when the protocol is already paused is meaningless, and the same applies to invoking the function `unpause()` when it is already active.

```
pub fn pause(&mut self) -> Result<()> {
    self.config_data.paused = true;

    emit!(ContractPaused {
        operator: self.signer.key(),
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}

pub fn unpause(&mut self) -> Result<()> {
    self.config_data.paused = false;

    emit!(ContractResumed {
        owner: self.signer.key(),
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}
```

Listing 2.9: programs/onchain_solana/src/instructions/pausable.rs

```
pub fn pause(&mut self) -> Result<()> {
    self.config_data.paused = true;

    emit!(ContractPaused {
        operator: self.signer.key(),
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}

pub fn unpause(&mut self) -> Result<()> {
    self.config_data.paused = false;

    emit!(ContractResumed {
        owner: self.signer.key(),
        timestamp: Clock::get()?.unix_timestamp,
    });

    Ok(())
}
```

Listing 2.10: programs/onchain_solana/src/instructions/pausable.rs

Suggestion Revise the logic to ensure function `pause()` only executes when not paused and function `unpause()` only executes when paused.

Feedback from the project This function is a privileged function, the team prevents misuse through off chain operational safeguards.

2.3 Note

2.3.1 Potential centralization risks

Introduced by `Version 1`

Description In this project, several privileged roles (e.g., `owner`) can conduct sensitive operations, which introduces potential centralization risks. For example, the `owner` can set `supported_aggregators` and `vault`. If the private keys of the privileged accounts are lost or maliciously exploited, it could pose a significant risk to the protocol.

Feedback from the project The team ensures the `owner` account is governed by a multisig wallet.

2.3.2 Ensure off-chain logic is trusted

Introduced by `Version 1`

Description The functionality of the contract highly depends on the off-chain logic. Thus, the off-chain part should be trusted. For example, the project allows the privileged `owner` to update supported aggregators via the function `manage_aggregators()`. These aggregators are outside the scope of this audit, and we assume they are compatible with the current implementation. However, in some aggregator designs, a swap may succeed under the slippage constraint while only partially consuming the provided `amount_in`. In such cases, the unused tokens remain untracked on-chain, and their proper refund depends entirely on off-chain handling. This design assumes that the off-chain logic is both correct and trusted.

Feedback from the project The team secures the off-chain logic and reconciles it against on-chain event logs to ensure the protocol functions as intended.

BOOST WEB3 THROUGH NEXT-GENERATION SECURITY & USABILITY INNOVATIONS